CyberMorph System Architecture

CyberMorph combines offline mobile gameplay with an online account and progress portal. The architecture connects its runtime services, AI research process, and software delivery workflow.

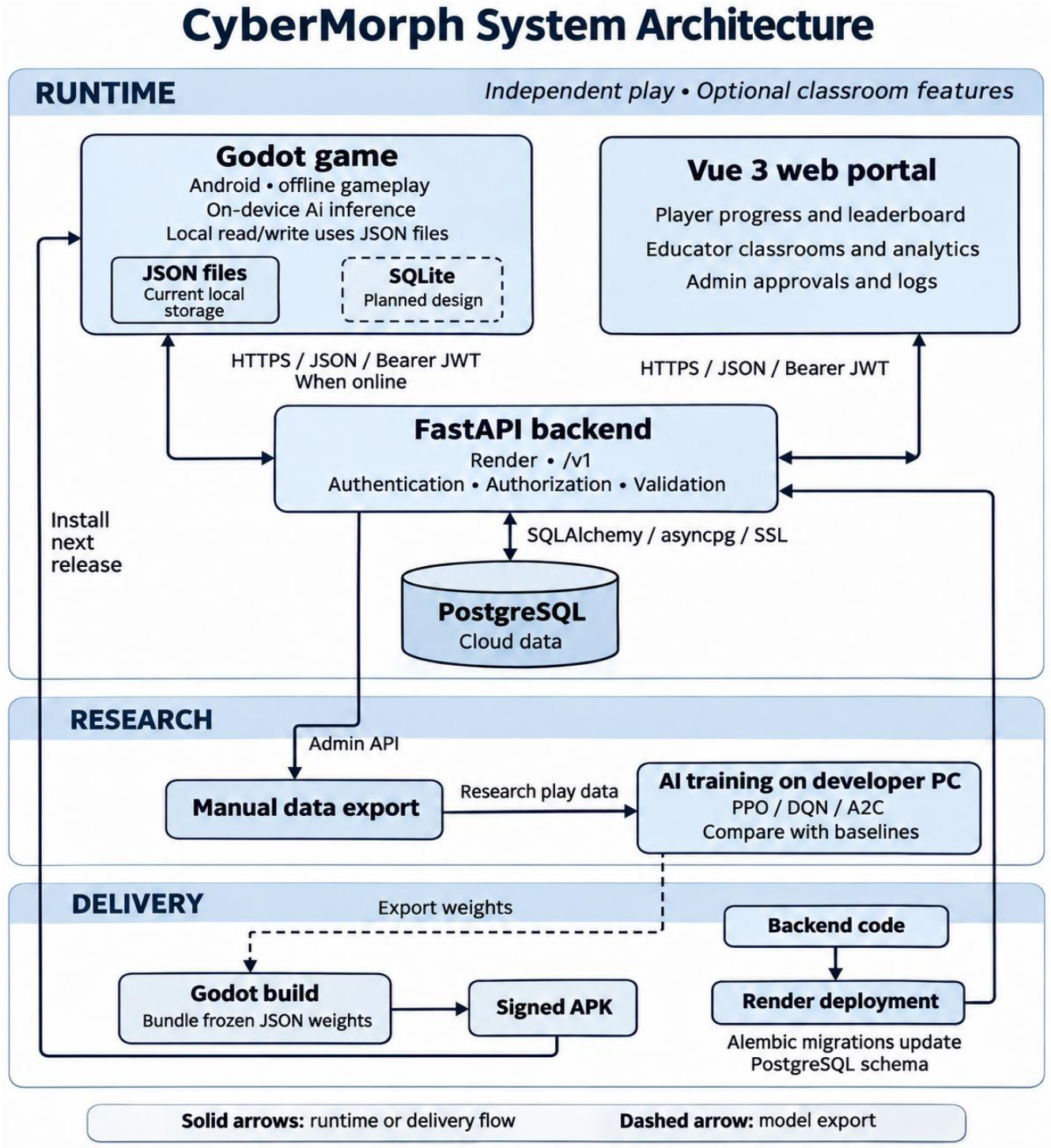


Figure 1. Runtime, research, and delivery architecture. SQLite is a planned design; the built game uses JSON files. The dashed arrow represents model export, and the dashed storage border identifies planned SQLite.

Components and Data Flow

The Godot Android game is the primary gameplay client. It runs game logic and the bundled AI model on the device, allowing independent play without a network connection. Classroom participation is optional.

The Vue 3 web portal provides Player progress and leaderboard views, Educator classroom management and student analytics, and Admin approval and log views. Vue Router handles navigation, Pinia manages application state, and dedicated API modules communicate with FastAPI.

FastAPI on Render exposes the /v1 API and handles authentication, authorization, input validation, and server-side processing. It accesses PostgreSQL through SQLAlchemy asynchronous sessions and asyncpg over SSL. Neither the mobile client nor the portal accesses the cloud database directly.

Offline Storage and Synchronization

The built game stores local records in JSON files under Godot's user:// directory, including guest and player records, pending runs, and authentication data. Runs are saved locally first. SQLite is the planned alternative for local gameplay records and a pending synchronization queue; it is not shown as an implemented database.

When online and authenticated, the game can submit sessions and supported synchronization records to FastAPI. The backend validates submitted data, persists it in PostgreSQL, and returns relevant progress information. The portal reads synchronized cloud records through its API endpoints; its views are not a live feed of offline play.

Research and Model Delivery

Research play data is uploaded through the game's play-data API and retrieved through authorized Admin API access. A manual export supplies the developer-side training pipeline. PPO, DQN, and A2C models are compared with baselines, and selected frozen weights are exported as JSON for bundling into a later Godot build.

The signed APK carries the model to players' phones. Inference runs locally during play; training occurs separately on the developer PC. Backend code is deployed to Render, while Alembic migrations manage PostgreSQL schema changes.

Authentication and Security Boundaries

Client API communication uses HTTPS and JSON, with bearer JWTs on protected requests. FastAPI enforces user roles and resource access. Frontend route guards guide navigation but are not a security boundary. Educator accounts require Admin approval before portal access.